

RAJALAKSHMI ENGINEERING COLLEGE

B. TECH – INFORMATION ECHNOLOGY

LAB MANUAL

Regulation 2023

AI23231 - PRINCIPLES OF ARTIFICIAL INTELLIGENCE

V SEM – III YEAR

Subject Code	Subject Name (Lab oriented Theory Course)	Category	L	T	P	C
AI23231	PRINCIPLES OF ARTIFICIAL INTELLIGENCE	PC	3	0	2	4

Objectives:
● Understand the various characteristics of a problem solving agent
● Learn about the different strategies involved in problem solving
● Learn about solving problems with various constraints.
● Apply A.I to various applications like expert systems etc.
● Understand the different models of learning

UNIT-I	**Introduction to Artificial intelligence and Problem-Solving Agent**	**9**
Problems of AI, AI technique, Tic – Tac – Toe problem. Intelligent Agents, Agents & environment, nature of environment, structure of agents, goal-based agents, utility-based agents, learning agents. Defining the problem as state space search, production system, problem characteristics, issues in the design of search programs.		
UNIT-II	**Search techniques**	**9**
Problem solving agents, searching for solutions; uniform search strategies: breadth first search, depth first search, depth limited search, bidirectional search, comparing uniform search strategies. Heuristic search strategies Greedy best -first search, A* search, AO* search, memory bounded heuristic search: local search algorithms & optimization problems: Hill climbing search, simulated annealing search, local beam search.		
UNIT-III	**Constraint satisfaction problems and Game Theory**	**9**
Local search for constraint satisfaction problems. Adversarial search, Games, optimal decisions & strategies in games, the minimax search procedure, alpha-beta pruning, additional refinements, iterative deepening.		
UNIT-IV	**Knowledge & reasoning**	**9**
Statistical Reasoning: Probability and Bays‟ Theorem, Certainty Factors and Rule-Base Systems, Bayesian Networks, Dempster-Shafer Theory, Fuzzy Logic. AI for knowledge representation, rule-based knowledge representation, procedural and declarative knowledge, Logic programming, Forward and backward reasoning.		
UNIT-V	**Introduction to Machine Learning**	**9**
Exploring sub-discipline of AI: Machine Learning, Supervised learning, Unsupervised learning, Reinforcement learning, Classification problems, Regression problems, Clustering problems, Introduction to neural networks and deep learning.		
	Contact Hours : 45	

List of Experiments				
1	**Programs on Problem Solving**			
a	Write a program to solve 8 Queens problem.			
b	Solve any problem using depth first search.			
c	Implement MINIMAX algorithm.			
d	Implement A* algorithm			
2	**Programs on Decision Making and Knowledge Representation**			
a	Introduction to PROLOG			
b	Implementation of Unification and Resolution Algorithm.			
c	Implementation of Backward Chaining			
d	Implementation of Forward Chaining			
3	**Programs on Planning and Learning**			
a	Implementation of Blocks World program			
b	Implementing a fuzzy inference system			
c	Implementing Artificial Neural Networks for an application using python			
d	Implementation of Decision Tree			
e	Implementation of K-mean algorithm			
		Contact Hours	**:**	**30**
		Total Contact Hours	**:**	**75**

Lab Specifications:

- The lab can be implemented using Python or C.
- Knowledge representation experiments can be performed using a PROLOG TOOL.

Course Outcomes: On completion of the course, the students will be able to
Basic knowledge representation, problem solving, and learning methods of artificial intelligence.
Provide the apt agent strategy to solve a given problem
Represent a problem using first order and predicate logic
Design applications like expert systems and chat-bot.
Suggest supervised, unsupervised or semi-supervised learning algorithms for any given problem

Text Books(s):
1. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Prentice Hall, Third Edition, 2015.
2. Nils J. Nilsson, Artificial Intelligence: A New Synthesis (1 ed.), Morgan-Kaufmann, 1998. ISBN 978- 1558605350.
Reference Book(s) / Web link(s):
1. Elaine Rich, Kevin Knight, &Shivashankar B Nair, Artificial Intelligence, McGraw Hill, 3rd ed.,2017.
2 .Introduction to Artificial Intelligence & Expert Systems, Patterson, Pearson, 1st ed. 2015
3. Logic & Prolog Programming, Saroj Kaushik, New Age International, Ist edition, 2002.
4. Expert Systems: Principles and Programming,11 March 1998. Edition: 4th. ISBN: 9788131501672

CO - PO – PSO Mapping

PO/PSO / CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
AI19341.01	3	3	1	-	2	1	1	1	1	-	2.2	1	2	1	1
AI19341.02	2	2	1	-	2	1	2	-	-	-	2	2	1	1	1
AI19341.03	3	3	1	-	3	-	1	-	-	-	3	1	2	3	2
AI19341.04	2	3	-	-	2	1	1	1	-	-	2	2	2	2	3
AI19341.05	2	2	2	2	3	-	1	2	-	-	3	3	3	3	3
Average Mapping	**2.4**	**2.4**	**1.25**	**2.0**	**2.4**	**1.5**	**1.2**	**1.3**	**1.0**	**-**	**2.4**	**1.8**	**2.0**	**2.0**	**2.0**

Note: Enter correlation levels 1, 2 or 3 as defined below:
1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)

Programs on Problem Solving

a: Solve the 8 Queens Problem

Aim

To place 8 queens on an 8x8 chessboard so that no two queens attack each other, i.e., no two queens share the same row, column, or diagonal.

Explanation:

This is a classic **Constraint Satisfaction Problem (CSP)**. We solve it using **backtracking**, which means placing queens row by row, checking safety, and backtracking when a conflict arises.

Procedure:

1. Create an 8x8 board initialized with 0s.
2. Place a queen in each row by checking if it's safe.
3. If no column is safe in a row, backtrack.
4. Repeat until all 8 queens are placed.

Python Code:

```
N = 8

def print_solution(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print("\n")

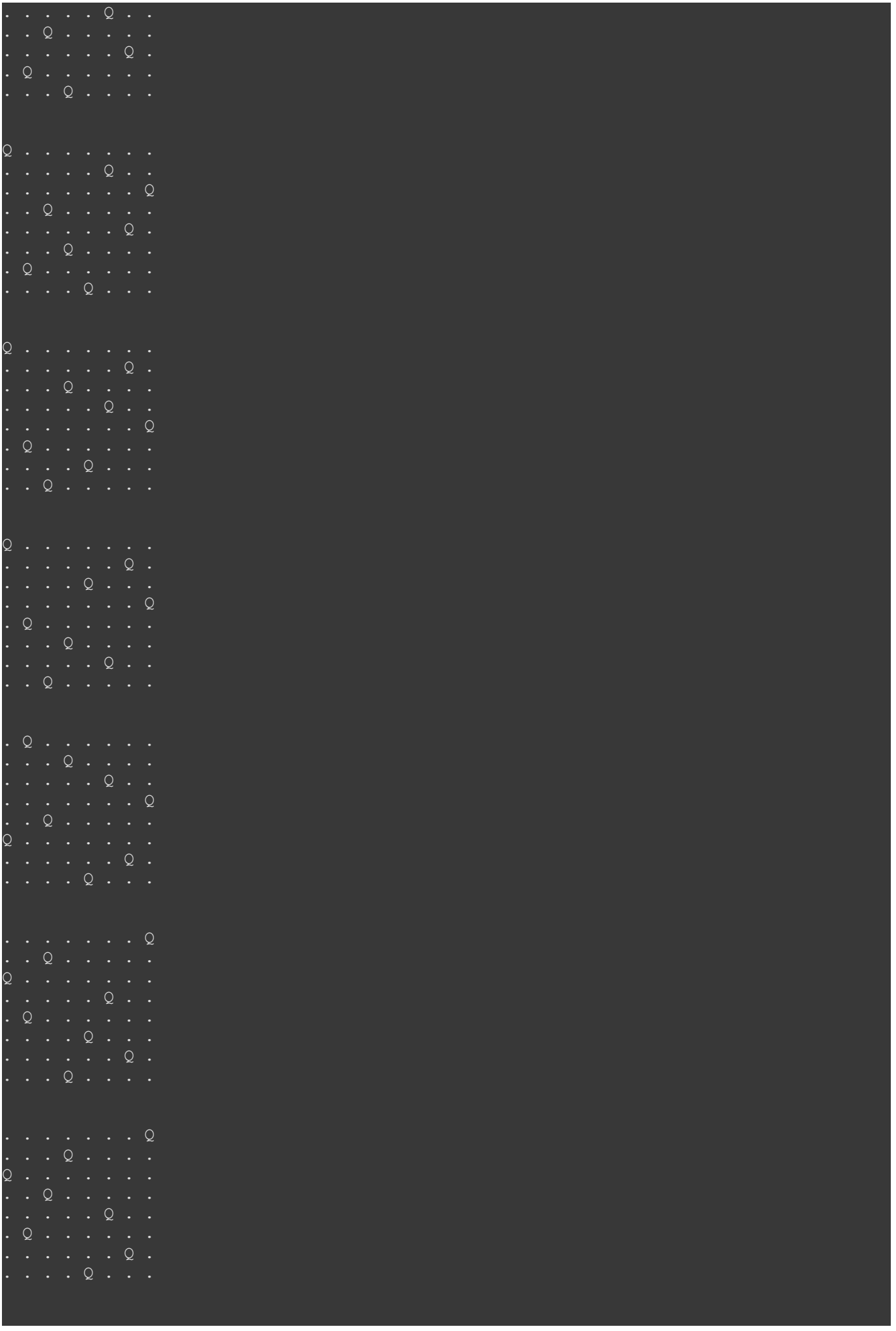
def is_safe(board, row, col):
    for i in range(row):
        if board[i][col] or \
            (col - (row - i) >= 0 and board[i][col - (row - i)]) or \
            (col + (row - i) < N and board[i][col + (row - i)]):
            return False
    return True

def solve_n_queens(board, row):
    if row == N:
        print_solution(board)
        return True
    found = False
    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            found = solve_n_queens(board, row + 1) or found
            board[row][col] = 0
    return found

board = [[0] * N for _ in range(N)]
solve_n_queens(board, 0)
```

Output :

```
. . . . . . . .
. . . . Q . . .
. . . . . . . Q
```



```
True
```

Result:

Thus the program of placing 8 queens on an 8x8 chessboard was executed successfully.

b: Solve a Graph using Depth First Search (DFS)

Aim :

To Implement DFS traversal on a graph to explore all reachable nodes from a source.

Explanation:

DFS explores nodes as deep as possible before backtracking. It uses recursion or an explicit stack.

Procedure:

1. Define the graph using an adjacency list.
2. Start from the source node.
3. Recursively visit all unvisited neighbors.

Python Code:

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# Sample graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

print("DFS traversal from node A:")
dfs(graph, 'A')
```

Output:

```
DFS traversal from node A:
A B D E F C
```

Result:

Thus the program of implementing DFS traversal on a graph to explore all reachable nodes from a source. was executed successfully.

c: Implement MINIMAX Algorithm

Aim :

To Use the MINIMAX algorithm to evaluate the optimal move in a game tree.

Explanation:

MINIMAX is a recursive algorithm used in decision-making and game theory. One player tries to maximize the score, while the other tries to minimize it.

Procedure:

1. Define a list of terminal values (leaf nodes).
2. Apply recursive MINIMAX from root to leaves.
3. Alternate between max and min at each level.

Python Code:

```
def minimax(depth, nodeIndex, isMax, scores, h):
   if depth == h:
      return scores[nodeIndex]

   if isMax:
      return max(minimax(depth + 1, nodeIndex * 2, False, scores, h),
                 minimax(depth + 1, nodeIndex * 2 + 1, False, scores, h))
   else:
      return min(minimax(depth + 1, nodeIndex * 2, True, scores, h),
                 minimax(depth + 1, nodeIndex * 2 + 1, True, scores, h))

scores = [3, 5, 6, 9, 1, 2, 0, -1]
h = 3
print("Optimal value:", minimax(0, 0, True, scores, h))
```

Output:

```
Optimal value: 5
```

Result:

Thus the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree was executed successfully.

d: Implement A*Algorithm

Aim :

To Find the shortest path from a source to a destination using the A* algorithm with heuristics.

Explanation:

A* is a graph search algorithm that finds the shortest path using both actual cost and estimated cost (heuristic).

Procedure:

1. Initialize a priority queue with the start node.
2. Maintain g(n) (path cost) and h(n) (heuristic).
3. Expand the lowest f(n) = g(n) + h(n) node.
4. Track path using came_from dictionary.

Python Code:

```
from queue import PriorityQueue

def a_star(graph, start, goal, heuristic):
    open_set = PriorityQueue()
    open_set.put((0, start))
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    while not open_set.empty():
        _, current = open_set.get()

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for neighbor, cost in graph[current]:
            tentative_g = g_score[current] + cost
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f = tentative_g + heuristic[neighbor]
                open_set.put((f, neighbor))
    return None

# Define graph and heuristic
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 1)],
    'D': [('G', 2)],
    'E': [('G', 1)],
    'F': [('G', 3)],
    'G': []
}
```

```
heuristic = {
    'A': 7, 'B': 6, 'C': 2,
    'D': 1, 'E': 0, 'F': 3, 'G': 0
}

path = a_star(graph, 'A', 'G', heuristic)
print("Shortest path using A*:", path)
```

Output:

```
Shortest path using A*: ['A', 'B', 'D', 'G']
```

Result:

Thus the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree was executed successfully

Programs on Decision Making and Knowledge Representation

a) Introduction to PROLOG

PROLOG (PROgraming in LOGic)

• PROLOG is declarative- specify facts and logical relationships

- Non-procedural
- Data base language with automated search and the ability to follow general rules.
- Symbolic - symbols are used to represent object
- High level - contains a built-in problem solving mechanism

1. PROLOG Programming (When we program in Prolog we need to provide following three things declaring some facts about objects and their relationships.
2. Defining some rules about objects and their relationships.
3. Asking questions about objects and their relationships.

Prolog Syntax

Terms:

The central data structure in PROLOG is that of a term. There are terms of four kinds:

- atoms,
- numbers,
- variables, and
- Compound terms.

Atoms and numbers are sometimes grouped together and called atomic terms.

Atomic Terms:

Atomic terms are usually strings made up of lower- and uppercase letters, digits, and the underscore, starting with a lowercase letter.

The following are all valid PROLOG atoms:

- elephant,
- b,
- abcXYZ,
- x_123,

Compound Terms:

- Compound terms are made up of a PROLOG atom and a number of arguments (PROLOG terms, i.e., atoms, numbers, variables, or other compound terms) enclosed in parentheses and separated by commas.
- The following are some examples for compound terms:
  - o is_bigger(horse, X), f(g(X, _), 7),
- The sets of compound terms and atoms together form the set of PROLOG predicates.
- A term that doesn't contain any variables is called a ground term.

Variables:

Variables are strings of letters, digits, and the underscore, starting with a capital letter or an underscore.

Examples:

- X,
- Elephant,
- _4711,
- X_1_2,
- MyVariable,

Facts:

A fact is a predicate followed by a dot.

Examples:

- bigger_animal(whale).
- life_is_beautiful.
- The intuitive meaning of a fact is that we define a certain instance of a relation as being true.

Rules:

- A rule consists of a head (a predicate) and a body (a sequence of predicates separated by commas).
- Head and body are separated by the sign :- and, like every PROLOG expression, a rule has to be terminated by a dot.

Examples:

is_smaller(X, Y) :- is_bigger(Y, X).

aunt(Aunt, Child) :- sister(Aunt, Parent),parent(Parent, Child).

- The intuitive meaning of a rule is that the goal expressed by its head is true, if we (or rather the PROLOG system) can show that all of the expressions (subgoals) in the rule's body are true.

Syntax of rule:

<head> :- <body>

Read :- as if.

->likes(john,X) :- likes(X,cricket).

~ "John likes X if X likes cricket".

i.e., "John likes anyone who likes cricket".

• Rules always end with'.'

Programs and Queries:

Programs-

A PROLOG program is a sequence of clauses.

Queries –

- After compilation a PROLOG program is run by submitting queries to the interpreter.
- A query has the same structure as the body of a rule, i.e. it is a sequence of predicates separated by commas and terminated by a dot.
- They can be entered at the PROLOG prompt, which in most implementations looks something like this: ?-.
- When writing about queries we often include the ?.
- Examples:

?- is_bigger(elephant, donkey).

?- small(X), green(X), slimy(X).

- Intuitively, when submitting a query like the last example, we ask PROLOG whether all its predicates are probably true, or in other words whether there is an X such that small(X), green(X), and slimy(X) are all true.

There are two types of queries:

1) Ground Query

- edge(a,h)
- This query is called a ground query because it consists only of value identifier as parameter to the predicate.
- A ground query is posed we expect a yes/no answer.

2) Non-Ground Query

They have variables as parameters

- edge(a,X)

Variables:

- Always begin with a capital letter
- ?- likes (john,X).
- ?- likes (john, Something).
- But not
- ?- likes (john,something)

Install SWI-Prolog:

Steps to Compile:

1. Open Notepad.
2. Write your Facts and Rules and save the file with .pl extension.
3. To open the desired file

   File -> Edit -> Go to the location-> select file ->Open

4. To check the syntax

In file editor

Compile ->make If

no error it will

compile

5. To fire query

In PROLOG editor

File -> consult -> select the file name

Write the query, do not forget to put full stop (.) If there is more than one answer to get use (;)

Exercise 1: Family Relationships

Problem: Define family relations and answer grandparent queries.

```
% Facts
parent(john, mary).
parent(mary, susan).
parent(john, mike).

% Rules
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

% Query
% ?- grandparent(john, susan).
```

Output:

```
true
```

Exercise 2: Animal Classification

Problem: Identify if an animal is a mammal based on features.

```
% Facts
has_hair(tiger).
gives_milk(tiger).

% Rule
mammal(X) :- has_hair(X), gives_milk(X).

% Query
% ?- mammal(tiger).
```

Output:

```
true.
```

Exercise 3: Simple Arithmetic Knowledge Base

Problem: Implement simple addition using PROLOG.

```
% Rule
add(X, Y, Z) :- Z is X + Y.

% Query
% ?- add(5, 7, Z).
```

Output:

```
Z = 12.
```

---

b) Unification and Resolution – Exercises with Solutions

Exercise 1: Sentence Matcher

```
% Facts
sentence(the_cat_sits).
sentence(the_dog_runs).

% Query
% ?- sentence(X).
```

Output:
X = the_cat_sits ; X = the_dog_runs ; false.

Exercise 2: Compound Term Unification

```
% Facts
likes(john, food(pizza, spicy)).
likes(mary, food(pizza, sweet)).

% Query
% ?- likes(john, food(pizza, T)).
```

Output:
T = spicy.

Exercise 3: Conflict Resolution

```
% Facts
rule1(climate_change).
rule2(climate_change).

% Rule
conflict(X) :- rule1(X), rule2(X).

% Query
% ?- conflict(climate_change).
```

Output:
true.

---

c) Backward Chaining

Exercise 1: Medical Diagnosis

```
% Facts
has_fever(john).
has_cough(john).

% Rule
has_flu(X) :- has_fever(X), has_cough(X).

% Query
% ?- has_flu(john).
```

Output:
true.

Exercise 2: Student Eligibility Checker

```
% Facts
passed(alice, maths).
```

```
passed(alice, science).

% Rule
eligible(X) :- passed(X, maths), passed(X, science).

% Query
% ?- eligible(alice).
```

Output:

```
true.
```

Exercise 3: Travel Planner

```
% Facts
has_ticket(james).
has_visa(james).

% Rule
can_travel(X) :- has_ticket(X), has_visa(X).

% Query
% ?- can_travel(james).
```

Output:

```
true.
```

---

d) Forward Chaining

Exercise 1: Simple Inference

```
:- dynamic known/1.

% Initial knowledge
start :- assert(known(a)), assert(known(b)), infer.

% Rule
infer :- known(a), known(b), \+ known(c), assert(known(c)), write('Inferred c'), nl.

% Query
% ?- start.
```

Output:

```
Inferred c
```

Exercise 2: Fire Alert System

```
:- dynamic known/1.

% Rule to infer fire
check :- known(smoke), known(heat), assert(known(fire)), write('Fire Alert!'), nl.

% Start rule
start :- assert(known(smoke)), assert(known(heat)), check.
```

% Query
% ?- start.

Output:

Fire Alert!

Exercise 3: Water Purity Detection

```
:- dynamic known/1.

% Rule to infer pollution
infer_pollution :- known(cloudy_water), known(bad_smell), assert(known(polluted)), write('Water is polluted'), nl.

start :- assert(known(cloudy_water)), assert(known(bad_smell)), infer_pollution.

% Query
% ?- start.
```

Output:

Water is polluted

Programs on Planning and Learning

a) Implementation of Blocks World Program

Problem Statement:
Simulate the Blocks World problem where a robot moves blocks from one configuration to another using planning.

Explanation:
Blocks World is a classical planning problem. The objective is to rearrange a set of blocks using rules (like picking up, putting down, and stacking).

Procedure:

1. Define the initial and goal states.
2. Apply rules for block movement.
3. Use recursive planning.

Python Code :

```
initial_state = [['A', 'B'], ['C'], []]
goal_state = [['C', 'A'], ['B'], []]

def move(state, from_stack, to_stack):
    state[to_stack].append(state[from_stack].pop())
    return state

def display_state(state):
    for i, stack in enumerate(state):
        print(f"Stack {i+1}: {stack}")
    print()

print("Initial State:")
display_state(initial_state)

# Simulate moving blocks manually (in real planning, use search)
state = move(initial_state, 1, 2)  # Move C
state = move(state, 0, 1)          # Move B
state = move(state, 0, 1)          # Move A
state = move(state, 1, 0)          # Move B
state = move(state, 2, 0)          # Move C

print("Final State:")
display_state(state)
```

Output:

```
Initial State:
Stack 1: ['A', 'B']
Stack 2: ['C']
Stack 3: []

Final State:
Stack 1: ['A', 'C']
Stack 2: ['B']
```

```
Stack 3: []
```

b) **Implementing a Fuzzy Inference System**

Problem Statement:
Create a fuzzy logic system to evaluate temperature and output fan speed.

Explanation:
Fuzzy logic handles approximate reasoning. It uses membership functions and fuzzy rules.

Procedure:

1. Define input/output variables.
2. Set membership functions.
3. Create rules and defuzzify output.

Python Code (using skfuzzy):

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# Input and Output variables
temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')

# Membership functions
temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 20])
temperature['warm'] = fuzz.trimf(temperature.universe, [10, 20, 30])
temperature['hot'] = fuzz.trimf(temperature.universe, [20, 40, 40])

fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25, 50, 75])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])

# Rules
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])

# Control system
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)

# Input and compute
fan_sim.input['temperature'] = 30
fan_sim.compute()
print("Fan Speed:", fan_sim.output['fan_speed'])
```

Output:

```
Fan Speed: 80.55555555555556
```

c) Implementing Artificial Neural Networks (ANN)

Problem Statement:
Build a simple neural network to classify handwritten digits (MNIST).

Explanation:
ANNs are inspired by the brain and are used for classification, prediction, and learning.

Procedure:

1. Load data.
2. Define a neural network.
3. Train and evaluate.

Python Code (using Keras):

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Load data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train/255.0, x_test/255.0

# One-hot encode labels
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Define ANN
model = Sequential([
   Flatten(input_shape=(28, 28)),
   Dense(128, activation='relu'),
   Dense(10, activation='softmax')
])

# Compile & Train
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, validation_split=0.1)

# Evaluate
loss, acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", acc)
```

Output:

```
Downloading data from
https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py
:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)` object as
the first layer in the model instead.
  super().__init__(**kwargs)
Epoch 1/5
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8642
- loss: 0.4699 - val_accuracy: 0.9648 - val_loss: 0.1215
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9611
- loss: 0.1288 - val_accuracy: 0.9690 - val_loss: 0.1027
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9755
- loss: 0.0857 - val_accuracy: 0.9747 - val_loss: 0.0867
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9822 -
loss: 0.0614 - val_accuracy: 0.9755 - val_loss: 0.0840
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9871
- loss: 0.0443 - val_accuracy: 0.9765 - val_loss: 0.0797
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9730 -
loss: 0.0902
Test Accuracy: 0.9757000207901001
```

d) **Implementation of Decision Tree**

Problem Statement:
Train a decision tree to classify iris flower species.

Explanation:
A decision tree splits data based on feature thresholds and learns interpretable rules.

Procedure:

1. Load dataset.
2. Train the Decision Tree classifier.
3. Test and print accuracy.

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy: 0.8666666666666667
```

e) **Implementation of K-Means Algorithm**

Problem Statement:
Use K-Means to cluster data points into groups (e.g., customers or flowers).

Explanation:
K-Means clusters data into K groups based on distance minimization.

Procedure:

1. Load data.
2. Fit KMeans.
3. Plot or evaluate clusters.

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load data
data = load_iris()
X = data.data

# Apply KMeans
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)
labels = kmeans.labels_

# Plot (2 features only for visualization)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='rainbow')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering")
plt.show()
```

Output:

